

Architecture And Flow Document

Project: AI-powered Data Analytics Application

1. Project Overview

This project is a full-stack analytics application that allows users to ask natural-language questions and get answers from structured dataset tables. It combines a React frontend, an Express backend, a SQLite database built from CSV files, and Groq for SQL generation.

Frontend: React + Vite + Tailwind

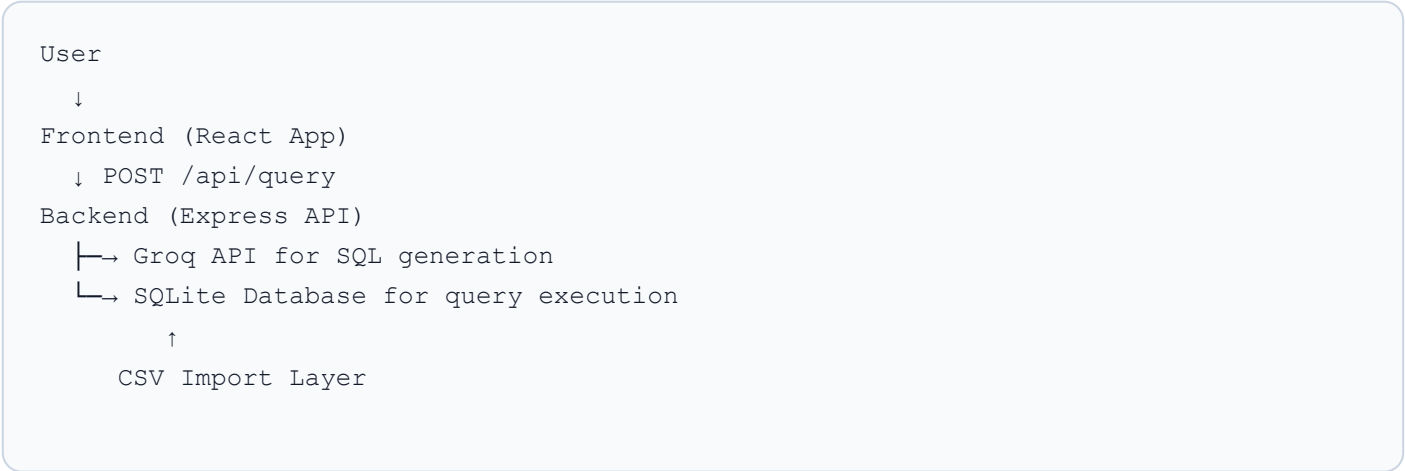
Backend: Node.js + Express

Database: SQLite via sql.js

AI: Groq

Deploy: Vercel

2. High-Level Architecture



3. Folder Structure

```
d:\Data-&-Analytics
├── frontend
│   ├── src
│   │   ├── App.jsx
│   │   ├── main.jsx
│   │   └── index.css
│   ├── package.json
│   └── vite.config.js
├── backend
│   ├── src
│   │   ├── app.js
│   │   └── server.js
```

```
| | | └─ config
| | | └─ controllers
| | | └─ database
| | |   └─ database.js
| | |   └─ csv
| | | └─ middlewares
| | | └─ prompts
| | | └─ routes
| | | └─ services
| | |   └─ utils
| └─ package.json
|   └─ .env
└─ .gitignore
└─ vercel.json
```

4. Frontend Architecture

Main Responsibilities

- Collect the user question
- Call backend API with Axios
- Show generated SQL
- Render results table
- Render chart when possible
- Show loading and error states

Important File

- **frontend/src/App.jsx**
- Main UI logic
- API request handling
- Response rendering
- Chart rendering

5. Backend Architecture

The backend follows a layered structure:

Route → Controller → Service → Database

Core Files

- **backend/src/app.js** for Express setup
- **backend/src/server.js** for startup and DB init
- **routes/query.routes.js** for API mapping

Service Files

- **sqlGenerator.service.js**
- **sqlValidator.service.js**
- **queryExecutor.service.js**

- `controllers/query.controller.js` for orchestration

- `chartDetector.service.js`
- `insightGenerator.service.js`

6. Backend Request Flow

1. User asks a question in the frontend
2. Frontend sends POST `/api/query` to backend
3. Backend reads the live database schema
4. Backend sends question + schema to Groq
5. Groq returns SQL
6. Backend validates the SQL
7. Backend runs the SQL on SQLite
8. Backend formats results, chart type, and insights
9. Backend sends JSON response to frontend
10. Frontend displays SQL, results, and chart

7. Database Flow

```
CSV Files
  ↓
Parse headers and rows
  ↓
Infer column types
  ↓
Create SQLite tables
  ↓
Insert records
  ↓
Export and persist nf_buildathon.db
```

Database Behavior

- Reads all CSV files from **backend/src/database/csv**
- Builds tables automatically from file names and headers
- Uses **sql.js** because native SQLite packages failed on Windows
- Stores the generated SQLite database as **nf_buildathon.db**

8. SQL Generation Pipeline

```
Question
  ↓
schemaPrompt.js builds live schema context
  ↓
sqlPrompt.js builds LLM prompt
  ↓
Groq generates SQL
  ↓
sqlGenerator.service.js extracts SQL
  ↓
sqlValidator.service.js blocks unsafe queries
  ↓
queryExecutor.service.js executes valid SQL
```

9. API Response Shape

```
{
  "success": true,
  "data": {
    "question": "show all users from Delhi",
    "sql": "SELECT * FROM users WHERE city = 'Delhi'",
    "validation": {
      "isValid": true,
      "warnings": [],
      "suggestions": []
    },
    "results": [],
    "chartType": "table",
    "insights": {
      "summary": "Query executed successfully",
      "keyPoints": []
    }
  }
}
```

10. Build Flow Followed

1. Created frontend and backend folders
2. Set up backend architecture
3. Added routes, controllers, services, middleware, and utils
4. Enabled ES modules
5. Tried native SQLite packages
6. Switched to sql.js after Windows/native install failures
7. Rebuilt database from CSV files
8. Added schema-aware SQL generation
9. Added SQL validation and execution
10. Changed AI provider from Gemini to xAI/Grok to Groq
11. Set up frontend with React + Vite + Tailwind
12. Connected frontend to backend
13. Added root .gitignore
14. Added Vercel deployment config
15. Fixed Vercel backend entrypoint

11. Deployment Architecture

Vercel

- └─ Frontend Service (root: frontend)
- └─ Backend Service (root: backend, entrypoint: src/server.js)

Frontend

- └─ calls Backend API

Backend

- └─ reads SQLite DB
- └─ calls Groq API

12. Key Decisions

- Used **sql.js** instead of native SQLite packages
- Built DB dynamically from CSV files
- Used schema-aware prompts for better SQL generation
- Restricted SQL to read-only queries

- Kept frontend simple first
- Moved to Groq after provider issues
- Added root-level deployment config
- Added root **.gitignore** for repo hygiene

13. Current Limitations

- Some files still use old `gemini` names even though Groq is used
- Chart detection is basic
- Insight generation is basic
- Frontend can be further componentized
- Production deployment may need more backend hardening

14. End-to-End User Journey

```
User opens frontend
↓
User enters analytics question
↓
Frontend sends request to backend
↓
Backend reads schema from SQLite
↓
Groq generates SQL
↓
Backend validates SQL
↓
Backend executes SQL
↓
Backend returns results and insights
↓
Frontend displays answer
```

15. Summary

This project was built as a complete AI-to-database analytics pipeline. CSV data becomes SQLite tables, user questions become SQL through Groq, SQL is validated and executed in the backend, and results are displayed in the frontend as tables and charts.